

This work is licensed under a Creative Commons license



Attribution-NonCommercial-NoDerivatives 4.0 International
(CC BY-NC-ND 4.0)

You are free to:

- **Share** – copy and redistribute the material in any medium or format.

Under the following terms:

- **Attribution** – You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** – You may not use the material for commercial purposes.
- **NoDerivatives** – If you remix, transform, or build upon the material, you may not distribute the modified material.

Automating Ghidra

Scripting with PyGhidra

Simone Aonzo, PhD



Last update: 2026-04-28T11:19:32

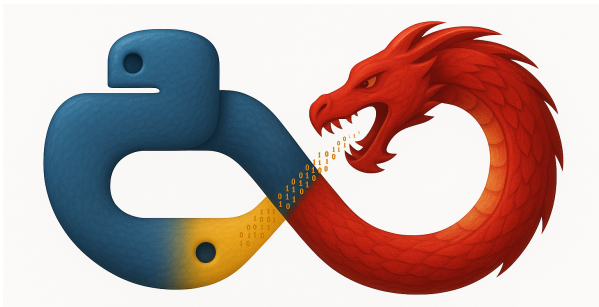
Scripting Ghidra [(NS25)] allows you to automate certain actions and, if necessary, run them on a large scale in **headless** mode (i.e., without a GUI)

Ghidra supports

- Java 🌿
- Python 2 through Jython (Java implementation of Python VM) 🌿
- **Python 3** (finally) officially supported 🇺🇸
 - but you need to start Ghidra via
 - `./support/pyghidraRun` in Linux
 - `.\support\pyghidraRun.bat` in Windows
 - otherwise you get *"Ghidra was not started with PyGhidra"*
 - start interpreter with: Window → PyGhidra

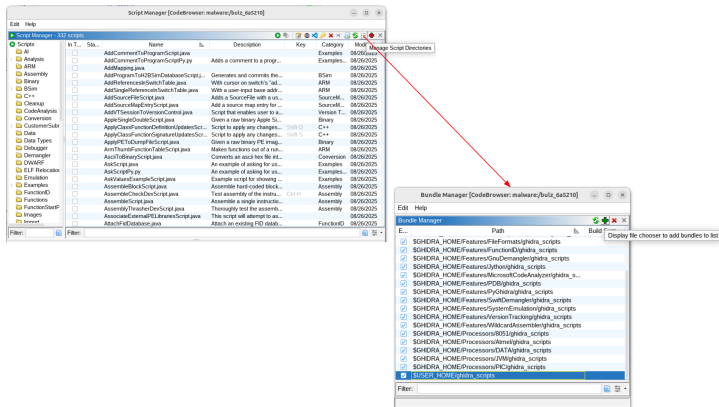
PyGhidra

PyGhidra is a thin **Python** layer over Ghidra + **JPy**



- Your **Python** code is running on **CPython**
- But the objects you manipulate live on the “**Java** heap” inside an embedded **JVM**
- **JPy** connects the two worlds wrapping **Java** objects in special **Python** proxy classes

Script Directory



Window → Script Manager

Irrespective of the language (Java/Python), all the scripts are there

Hello world from Python

Create New Script → PyGhidra

```
#This is the script description
#@author Simone Aonzo
#@category Examples.Python
#@keybinding ctrl alt shift H
#@menupath Tools.Hi from Py
#@toolbar
#@runtime PyGhidra

import sys
print("Hello from Python", sys.version)
```

Metadata:

- `@category` Category path; levels are separated by "."
- `@keybinding` case sensitive; e.g., L, or ctrl alt shift F12
- `@menupath` Level dot-separated, see next slide
- `@toolbar` icon, searches in script directories and then Ghidra

- Online: https://ghidra.re/ghidra_docs/api/
- Offline: unzip docs/GhidraAPI_javadoc.zip
 - → GhidraAPI_javadoc/api/

Mixing Java/Python can be tricky. Rule of thumb:

- Java: `some.long.package.ClassName`
- Python: `from some.long.package import ClassName`

At the very beginning, without the need of importing anything, you already have access to all the methods/fields of

`api/ghidra/app/script/GhidraScript`

Two Main APIs: the Program API and the Flat API

① Program API

- More complex and comprehensive
- May change from Ghidra version to version

② Flat Program|Decompiler API

- “Flattened” version that provides quick access to various layers
- Stable across Ghidra’s version $\implies$ use it!
- GhidraScript extends the Flat Program API

- `print()` – print to console
- `popup()` – GUI message box
- `ask...` – and validates input
 - `askString(title, msg)`
 - `askYesNo(title, msg)` — returns a boolean, True for yes
 - `askAddress(title, msg)`
 - `askInt(title, msg)`
 - `askFile(title, approve_btn_text)`
 - `askDirectory(title, approve_btn_text)`

they raise a `CancelledException` when the user cancels

GhidraScript – State

- `currentProgram` – the current active open program
 - type: `ProgramDB`, see [docs/api/ghidra/program/database/ProgramDB.html](https://ghidra-sre.org/docs/api/ghidra/program/database/ProgramDB.html)
 - many `get...` methods; e.g. `getExecutableEntryPoint()`
- `currentLocation` – the program location of the cursor
 - a `ProgramLocation`, the `currentLocation().getAddress()` is...
- `currentAddress` – the current address of the location of the cursor
 - this is an `Address`, in an address space
 - usually you just need the offset: `getOffset()`
 - `toAddr(offset)` returns an `Address`; e.g., `goTo(toAddr(0x1234))`
 - given an address, you can “move” with `next()/previous()/add(n)/subtract(n)`
- `currentSelection` – the current selection or `None`
 - a `ProgramSelection`, which allows us to iterate over `AddressRanges`, `getMinAddress()`, `getMaxAddress()`, ...

Long operations should check `getMonitor().isCancelled()`

```
for i in range(100):  
    if getMonitor().isCancelled():  
        print('Bye!')  
        break  
    print(i)  
    time.sleep(.5)
```

Memory

Reading

- `get{Byte,Int,Long}(addr)`
 - return *signed* values
 - to convert to unsigned, you can use: `... & 0xff`, `... & (2**32-1)` and `... & (2**64-1)`
- `getBytes(addr, len)`
 - returns a `jep.PyJArray`; you can use `bytes(b & 0xff for b in getBytes(...))` to get Python bytes

Writing

- `set{Byte,Int,Long}(addr, value)`
- `setBytes(addr, bytes)`

to change/edit existing data, you need to `clearListing` first

→ bulz_6a5210

- ① Complete `read_c_string`, in `01_bulz.py`, to read a C-string returned as Python bytes/bytearray, from a starting address `addr`
 - hint: create a `bytearray()`, and then append bytes to it
 - `read_c_string(toAddr(0x10002124))` should return a path
 - remember to check `getMonitor().isCancelled()` in your loop
- ② Reverse `FUN_10001210`; can you spot a string decryption routine?
 - How does it work?
 - Complete `02_bulz.py` by implementing the decryption routine in Python; then, check the result of the script

Code units (Data and Instructions)

- **Data** – `api/ghidra/program/model/listing/Data`
 - `getFirstData()`, `getLastData()`, ...
- **Instructions** – `api/ghidra/program/model/listing/Instruction`
 - `getFirstInstruction()`, `getInstructionAfter(inst)`, `getInstructionAt(addr)`, ...
 - **members:** `getNumOperands()`, `getOpObjects()`, `getPrevious()/getNext()`, `getFlowType()`, ...

`getOpObjects()` – objects used by this operand (e.g., Address, Scalar, Register, ...)

`getFlowType()` – how it flows from one instruction to the next

All **Code units** have `get...` functions

- `MinAddress`, `MaxAddress`, `Bytes`, `Length`, `MnemonicString`, `PrimarySymbol`, ...

Functions

- `getFirstFunction()`
- `getGlobalFunctions(name)` – all functions in the global namespace with the given name.
- `currentProgram.getFunctionManager()`
 - `api/ghidra/program/model/listing/FunctionManager`
 - `getFunctionAt(entryPoint)`
 - `getFunctions(True)` – iterates functions in address order
- `members:`
 - `api/ghidra/program/model/listing/Function`
 - `getName()`
 - `getEntryPoint()`
 - `getBody()`

Listing

Listing provides all the functionalities to operate on code level constructs

Example: iterate over all functions and their instructions

```
fm = currentProgram.getFunctionManager()
listing = currentProgram.getListing()

for f in fm.getFunctions(True):
    body = f.getBody()
    for i in listing.getInstructions(body, True):
        print(i)
```

[api/ghidra/program/model/listing/Listing](#)

→ `bulz_6a5210`

Complete `03_bulz.py` so that it:

- ① iterates over all functions, and for each
- ② counts the number of instructions (`count_instructions(func)`)
- ③ prints: name, address, and number of Instructions

What do you notice about the naming convention of the functions?

Symbols and References

- **Symbol**, lookup:

- `getSymbolAt(addr)`

members:

- `getName()`
- `getAddress()`

- **Reference**, lookup:

- `getReferencesFrom(addr)`
- `getReferencesTo(addr)`

members:

- `getFromAddress()`
- `getToAddress()`
- `getReferenceType()`, a `RefType`
 - `isCall()`, `isRead()`, ...

→ `bulz_6a5210`

Let's find all calls to `0x10001210`; complete `bulz_3.py` so that it:

- 1 gets all references to such an address, by implementing `get_references_to_decrypt_function`
 - use function `getReferencesTo`, then
 - filter out any reference that is not a call, if any (see `getReferenceType` and `isCall`)
- 2 prints, in hex, the address where each call takes place

Label, Bookmarks and Comments

- `createLabel(address, name, makePrimary)`
 - `makePrimary` is a boolean; the *primary* label is the one used to represent the address everywhere it is displayed
- `createBookmark(address, category, note)`
 - `category` can be `None`
- `setPreComment(addr, comment)`

```
; This function checks user input and validates bounds  
; It returns 0 on success, -1 on failure  
00401000  push    ebp
```

- `setEOLComment(addr, comment)`

```
00401000  cmp     eax, 5      ; max allowed attempts
```

→ `bulz_6a5210`

For each call to `0x10001210`, notice that its three arguments are PUSHed on the stack.

Complete `05_bulz.py`:

- 1 Implement `get_push_immediate()` that checks if the push instruction is a `Scalar` and returns it
- 2 After retrieving the three arguments, call the `decrypt_str` function to get the decoded bytes
- 3 `createLabel(address, name, True)` each string with its decrypted value. This will make it easier to understand the code.

[(NS25] National Security Agency (NSA).
Ghidra software reverse engineering framework, 2025.

<https://github.com/NationalSecurityAgency/ghidra>.